

what's the difference between compiled and interpreted languages and in this way what about Csharp?

1. اللغات المجمعّة (Compiled Languages)

المباشرة للكمبيوتر قبل (Machine Code / 0s and 1s) "يأخذ الكود كله على بعضه دفعة واحدة، ويترجمه لملف ثنائي خالص فيه "لغة الآلة" (Compiler) المترجم ما تنفذ أو تشغل البرنامج.

- **طريقة العمل:** التشغيل مباشر على المعالج → (Binary / .exe) ملف تنفيذي → (Compiler) المترجم → (Source Code) الكود
- **المميزات:** أداء وسرعة عالية جداً لأن الكود مترجم جاهز للمعيار بتاع جهازك.
- **العيوب:** للأسف الملف الناتج غير مرن؛ يعني الملف اللي اتقرأ واتجمع للوندوز مش هيشغل على الماك إلا لما تعيد تجميعه.
- كاملة قبل ما يخليك تشغل أي حاجة (Syntax Errors) **الأخطاء:** بيقفز ويكتشف الأخطاء البرمجية.
- **أمثلة:** C, C++, Rust, Go.

C

□□

```
// مثال كود C
#include <stdio.h>
int main() {
    printf("Hello, World!");
    return 0;
}
```

تشغل عليه بعدين exe. عشان يطالعك ملف gcc program.c -o program.exe في المثال ده: بتستخدم أمر زي

2. اللغات المفسرة (Interpreted Languages)

يأخذ الكود بتاعك يقرأه **سطر بسطر**، ويترجم كل سطر للغة الآلة وينفذه في نفس (Interpreter) هنا مفيش ملف تنفيذي بيتترجم مسبقاً. يبقى فيه برنامج تاني اسمه المفسر (At Runtime) اللحظة.

- **طريقة العمل:** يقرأ وينفذ سطر بسطر → المعالج (Interpreter) المفسر → (Source Code) الكود
- بشرط يكون المفسر مثبت على الجهاز (Cross-platform) **المميزات:** سهولة في التطوير وتعمل على أي نظام تشغيل.
- **العيوب:** أبطأ في التشغيل لأن الترجمة يتم وقت الرن (Real-time Overhead).
- **الأخطاء:** لو فيه خطأ في السطر رقم 10، البرنامج هيشغل عادي وينفذ أول 9 سطور وبعدين يهتق ويقف عند السطر الـ 10.
- **أمثلة:** Python, JavaScript, PHP, Ruby.

Python

□□

```
# مثال كود Python
print("Hello, World!")
```

ويقرأه فوراً ويعرضلك النتيجة بدون ما يحول لملف ثاني py. في المثال ده: المفسر بيأخذ أصل ملف الـ

3. (C#)? موقف لغة سي شارب

يتجمع بين الميزتين على مرحلتين (Hybrid Language) مش مجمعة تماماً ولا مفسرة تماماً، هي لغة هجينة **C#** لغة

- **المرحلة الأولى (Compilation):** اسمها IL (Intermediate Language / Bytecode) ويحوّله للغة **C#** بيأخذ كود (Roslyn Compiler) المترجم الخاص باللغة الكود ده مش لغة آلة لسه، لكنه لغة محايدة .exe أو .dll. وتحتفظها في ملفات زي
- **المرحلة الثانية (JIT / Just-In-Time):** بيحول الأجزاء المطلوبة **JIT Compiler** ده وبيستخدم مترجم اسمه IL بيأخذ كود الـ (CLR واسمه) .NET. لما بتيجي تشغل البرنامج، محرك البيئة الخاصة بـ حقيقي وينفذه فوراً على المعالج Machine Code فقط لـ.

[C# Code]

1. (Roslyn Compiler)

↓ (Roslyn Compiler)
[Intermediate Language (IL)]
↓ (CLR - JIT Compiler)
[Native Machine Code (0s and 1s)]

• بتعمل كده؟ **C#** ليه

وفي نفس الوقت تاخذ **السرعة العالية** القريبة جداً من لغات (.NET). يقدر يشتغل على أي نظام تشغيل وندوز/ماك/لينكس طالما فيه IL الكود الـ عشان تاخذ **المرونة** زي C++.

C#

□□

// مثال كود C#
using System;

```
class Program {
    static void Main() {
        Console.WriteLine("Hello, World!");
    }
}
```

بيحول له لكود آلة ويفعل البرنامج **JIT** مترجم الـ **dotnet run** ولما تعمل IL، بيطلع كود الـ **dotnet build** عند تشغيل

Compare between implicit, explicit, Convert and parse casting

(string أو double زي) وتحولها لنوع ثاني (int زي) معناه إنك بتاخذ قيمة من نوع بيانات معين (Type Casting / Conversion) تحويل أنواع البيانات، **C#** في لغة

عشان نفهم الفكرة بسهولة، تخيل إن أنواع البيانات عبارة عن **كاسات** ومجال أحجام

- **Implicit** (تلقائي / ضمني): بتعملها لوحدها وبأمان **C#** مش هيتلق نقطة ماء، فـ بتصب كاس صغير في كاس كبير: **(ضمني / تلقائي) Implicit**.
- **Explicit** ("تقولك": "تحمل أنت المسؤولية واكتب التحويل بنفسك **C#** ممكن ينزل ماء بره وتنفذ بيانات، فـ بتصب كاس كبير في كاس صغير: **(صريح / يدوي) Explicit**).
- **Parse**: جواه أرقام وتستخرج منه الرقم **(string)** بتاخذ نص: **Parse**.
- **Convert**: (null) الجوكرا! بيحول أي نوع لأي نوع ثاني بأمان وبيتعامل مع القيم الفاضية: **Convert**.

جدول مقارنة سريع

المفهوم	متى نستخدمه؟	بيحصل إزاي؟	نوع القيمة؟	لو التحويل فشل؟
Implicit	من نوع صغير لنوع كبير (int → double)	تلقائياً تلقائي بدون تدخل	لا ينطبق	لا يسبب أخطاء مسبقاً
Explicit	من نوع كبير لنوع صغير (double → int)	(type) يدوي باستخدام	لا ينطبق	(Truncate) بيحصل قطع للبيانات
Parse	فقط إلى رقم/تاريخ string تحويل	int.Parse(str)	يرمي استثناء (ArgumentNullException)	يرمي استثناء (FormatException)
Convert	تحويل بين مختلف الأنواع بمرونة	Convert.ToInt32(val)	يرجع 0 (أمن جداً)	يرمي استثناء (FormatException)

1. التحويل الضمني (Implicit Casting)

بيحصل لما تحول من نوع صغير إلى نوع أكبر منه في المساحة، أو من نوع غير صحيح لنوع عشري. التحويل ده آمن 100% لأن مقيش أي فقد للبيانات.

- الشكل: بيتم تلقائياً بدون ما تكتب أي أقواس أو أوامر.

C#

□□

```
int myInt = 10;
double myDouble = myInt; // implicit casting (int -> double)
```

Console.WriteLine(myDouble); // Output: 10

2. التحويل الصريح (Explicit Casting / Explicit Cast)

بتجبرك تكتب اسم النوع بين **C#** بيحصل لما تحول من نوع كبير إلى نوع أصغر منه في المساحة (زي تحويل رقم عشري لرقم صحيح). هنا في احتمالية تفقد جزء من البيانات (الكسور يتمسح)، وعشان كده عشان نتأكد إنك فاهم اللي بتعمله (type) أقواس

- الشكل: اسم النوع المطلوب بين أقواس قبل المتغير.

C#

```
double myDouble = 9.78;
int myInt = (int)myDouble; // explicit casting (double -> int)
```

Console.WriteLine(myInt); // Output: 9 (تم قطع الكسر 0.78 وفقدانه)

3. دالة Parse

(int.Parse, double.Parse, DateTime.Parse) زي لـ أي نوع ثاني **string** اختصاصها حاجة واحدة بس: تحويل الـ **Parse** الـ

- (Exception) يوقف التطبيق error فالبرنامج هيضرب null أو كان ("abc" زي) مش رقم حقيقي string عيبها الرئيسي: لو الـ

C#

```
string ageStr = "25";
int age = int.Parse(ageStr); // age = 25
```

ويوقف البرنامج Exception تنبيه: الكود اللي جاي هيضرب
// string badStr = "hello";
// int num = int.Parse(badStr);

لو التحويل فشل error وهي أفضل طريقة للتعامل مع مدخلات المستخدم لأنها مش بتضرب int.TryParse() بنستخدم بدالها Parse نصيحة للمحترفين: لو خايف الكود يضرب مع

4. كلاس Convert

(Convert.ToInt32, Convert.ToString, Convert.ToBoolean) هو الطريقة الأشمل والمرنة للتحويل بين الأنواع **Convert** فصل الـ

- (للبيانات false الـ 0 للأرقام أو) ولكن هترجع القيمة الافتراضية Parse زي error مش هتضرب null لو ادبتها قيمة. null ميزتها الذهبية: بتتعامل بذكاء مع الـ

C#

```
string numberStr = "100";
int num1 = Convert.ToInt32(numberStr); // num1 = 100
```

string nullStr = null;
int num2 = Convert.ToInt32(nullStr); // num2 = 0 (error! أمانة ومش بتضرب)